

P1040R6

std::embed

Published Proposal, 2020-02-17

Author:

[JeanHeyd Meneide](#)

Audience:

EWG, SG7

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Latest:

https://the-phd.github.io/vendor/future_cxx/papers/d1040.html

Implementation:

<https://github.com/ThePhD/embed>

Reply To:

[JeanHeyd Meneide](#), [@thephantomderp](#)

Abstract

A proposal for a function that allows pulling resources at compile-time into a program.

Table of Contents

1	Revision History
1.1	Revision 6 - March 2nd, 2020
1.2	Revision 5 - January 13th, 2020
1.3	Revision 4 - November 26th, 2018
1.4	Revision 3 - November 26th, 2018
1.5	Revision 2 - October 10th, 2018
1.6	Revision 1 - June 10th, 2018
1.7	Revision 0 - May 11th, 2018
2	Relevant Polls
3	Motivation
4	Scope and Impact
5	Design Decisions
5.1	Implementation Experience & Current Practice
5.1.1	Speed Results
5.1.2	Memory Size Results
5.1.3	Results Analysis
5.1.4	Manual Work
5.1.5	Processing Tools
5.1.6	<code>ld</code> , resource files, and other vendor-specific link-time tools
5.1.7	The <code>incbin</code> tool
5.2	Prior Art
5.2.1	Literal-Based, <code>constexpr</code>

- 5.2.2 Literal-Based, Null Terminated (?)
- 5.2.3 Encoding
- 5.3 Design Goals
 - 5.3.1 Implementation Defined
 - 5.3.2 Binary Only
 - 5.3.3 Constexpr Compatibility
 - 5.3.4 Dependency-Scanning Friendly with `#depend`
 - 5.3.5 Modules
 - 5.3.6 Statically Polymorphic
 - 5.3.7 Optional Limit
 - 5.3.8 UTF-8 Only

6 Previous Implementations

- 6.1 `#depend` Soft Warnings, Hard Errors?
- 6.2 String Table / Virtual File System

7 Changes to the Standard

- 7.1 Intent
- 7.2 Proposed Feature Test Macro
- 7.3 Proposed Wording

8 Appendix

- 8.1 Alternative
 - 8.1.1 Pre-Processing Tools Alternative
 - 8.1.2 `python` Alternative
 - 8.1.3 `ld` Alternative

9 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. Revision 6 - March 2nd, 2020

- Add new section [§ 2 Relevant Polls](#).
- Add new section [§ 5.3.4 Dependency-Scanning Friendly with `#depend`](#).
- Add new section [§ 5.3.5 Modules](#).
- Improve section [§ 5.3.6 Statically Polymorphic](#).
- Add new section [§ 5.3.7 Optional Limit](#).
- Add new section [§ 5.3.8 UTF-8 Only](#).
- Add new section [§ 6 Previous Implementations](#).
- Improve wording and add static version (thanks, @lichray).

§ 1.2. Revision 5 - January 13th, 2020

- Split `#embed` into a new paper.
- Add memory and time benchmarks from various implementation strategies in the [new Current Practice section](#).
- Address concerns for a generic API and similar in the [new Results Analysis section](#).
- Retarget to EWG and SG 7.

§ 1.3. Revision 4 - November 26th, 2018

- Wording is now relative to [\[n4778\]](#).
- Minor typo and tweak fixes.

§ 1.4. Revision 3 - November 26th, 2018

- Change to using `constexpr`.
- Discuss potential issues with accessing resources after full semantic analysis is performed. Prepare to poll Evolution Working Group. Reference new paper, [\[p1130\]](#), about resource management.

§ 1.5. Revision 2 - October 10th, 2018

- Destroy `embed_options` and `alignment` options: if the function is materialized only at compile-time through `constexpr` or the upcoming "immediate functions" (`constexpr!`), there is no reason to make this part of the function. Instead, the user can choose their own alignment when they pin this down into a `std::array` or some form of C array / C++ storage.

§ 1.6. Revision 1 - June 10th, 2018

- Create future directions section, follow up on Library Evolution Working Group comments.
- Change `std::embed_options::null_terminated` to `std::embed_options::null_terminate`.
- Add more code demonstrating the old way and motivating examples.
- Incorporate LEWG feedback, particularly alignment requirements illuminated by Odin Holmes and Niall Douglass. Add a feature macro on top of having `__has_include( <embed> )`.

§ 1.7. Revision 0 - May 11th, 2018

Initial release.

§ 2. Relevant Polls

The following polls are shaping the current design. Votes are in the form of SF (Strongly in Favor), F (in Favor), N (Neutral), A (Against), SA (Strongly Against).

`#depend` - We would like to have this feature in C++(Something) and spend time figuring out the details.

- SF F N A SA
- 14 13 2 0 0
- Consensus: Do more work.

`#depend <foo/**>` - We want recursive globs (recursively searching through directories) for `#depend`.

- SF F N A SA
- 2 3 9 12 4
- Consensus: Do not want.
- Vote Commentary:
- A: Complexity?
- SA: Windows is slow with recursive globs.

It should be mandatory that EVERY file for `std::embed` is specified by a `#depend`.

- SF F N A SA

- 4 12 5 6 3
- Consensus: Split, no consensus. Add why/why not.
- Vote Commentary:
- SF: This MUST exist. Both compiler and build system authors. (Implementers.)
- SA: Can make user experience sad face for common case. Build system should scream at you for making the mistake instead.

#depend should form a Virtual File System / String Table State that constrains the search and should be passed to std::embed.

- SF F N A SA
- 3 11 7 2 2
- Consensus: Do it.
- Vote Commentary:
- SA: Hell to implement. (This was the author.)

Make std::embed ill-formed inside of a module interface (with a plan to revisit later).

- SF F N A SA
- 4 2 7 1 1
- Consensus: Yes, but Meh.
- Vote Commentary:
- SA: Modules are important we should make sure it interacts well with modules (figure it out now).
- SF: How does this work with #depend ?
- SF: std::embed is basically a #include -- why would we want it in interface? Just focus on getting feature working and doing it well.
- A: Jumping the gun. Space needs more exploration.
- N: We are highly undecided - need to answer more questions (especially about Modules).

§ 3. Motivation

I’m very keen on std::embed. I’ve been hand-embedding data in executables for NEARLY FORTY YEARS now. — Guy "Hatcat" Davidson, June 15, 2018

Currently	With Proposal
<pre>se-shell@virt-deb> python strfy.py \ fxaa.spirv \ stringified_fxaa.spirv.h</pre>	

```

#include <span>

constexpr inline
const auto& fxaa_spirv_data =
#include "stringified_fxaa.spirv.h"
;

// prevent embedded nulls from
// ruining everything with
// char_traits<char>::length
// or strlen
template <typename T, std::size_t N>
constexpr std::size_t
string_array_size(const T (&)[N]) {
    return N - 1;
}

int main (int char*[]) {
    constexpr std::span<const std::byte>
    fxaa_binary{
        fxaa_spirv_data,
        string_array_size(fxaa_spirv_data)
    };

    // assert this is a SPIRV
    // file, at compile-time
    static_assert(fxaa_binary[0] == 0x03
        && fxaa_binary[1] == 0x02
        && fxaa_binary[2] == 0x23
        && fxaa_binary[3] == 0x07,
        "given wrong SPIRV data, "
        "check rebuild or check "
        "the binaries!")

    auto context = make_vulkan_context();

    // data kept around and made
    // available for binary
    // to use at runtime
    auto fxaa_shader = make_shader(
        context, fxaa_binary );

    for (;;) {
        // ...
        // and we're off!
        // ...
    }

    return 0;
}

```

```

se-shell@virt-deb> python gen_cxx_random_data.py \
-o include/gen/random_data.h

```

```

#include <embed>

int main (int, char*[]) {
    constexpr std::span<const std::byte>
    fxaa_binary =
        std::embed( "fxaa.spirv"

    // assert this is a SPIRV
    // file, at compile-time
    static_assert(fxaa_binary[0] == 0
        && fxaa_binary[1] == 0x02
        && fxaa_binary[2] == 0x23
        && fxaa_binary[3] == 0x07
        "given wrong SPIRV data, "
        "check rebuild or check "
        "the binaries!")

    auto context = make_vulkan_context();

    // data kept around and made
    // available for binary
    // to use at runtime
    auto fxaa_shader = make_shader(
        context, fxaa_binary );

    for (;;) {
        // ...
        // and we're off!
        // ...
    }

    return 0;
}

```

```

#include <stdint>
#include <utility>

constexpr std::uint64_t val_64_const
    = 0xcbf29ce484222325u;
constexpr std::uint64_t prime_64_const
    = 0x100000001b3u;

inline constexpr std::uint64_t
hash_64_fnv1a_const(const char* const ptr,
                    std::size_t ptr_size,
                    const std::uint64_t value = val_64_const
) noexcept {
    return (ptr_size == 1)
        ? value
        : hash_64_fnv1a_const(
            &ptr[1],
            ptr_size - 1,
            (value ^ static_cast<std::uint64_t>(*ptr))
            * prime_64_const
        );
}

#include <gen/random_data.h>

int main () {

    constexpr std::uint64_t actual
        = hash_64_fnv1a_const(&random_data[0],
                             std::size(random_data));

    return static_cast<int>(actual);
}

```

```

#include <embed>
#include <stdint>

constexpr std::uint64_t val_64_const
    = 0xcbf29ce484222325u;
constexpr std::uint64_t prime_64_const
    = 0x100000001b3u;

inline constexpr std::uint64_t
hash_64_fnv1a_const(const char* const ptr,
                    std::size_t ptr_size,
                    const std::uint64_t value = val_64_const
) noexcept {
    return (ptr_size == 1)
        ? value
        : hash_64_fnv1a_const(
            &ptr[1],
            ptr_size - 1,
            (value ^ static_cast<std::uint64_t>(*ptr))
            * prime_64_const
        );
}

int main () {
    constexpr std::span<const char> a
        = std::embed("/dev/urandom");
    constexpr std::uint64_t actual
        = hash_64_fnv1a_const(a,
                             a.size());

    return static_cast<int>(actual);
}

```

(Works here.)

A very large amount of C and C++ programmer -- at some point -- attempts to `#include` large chunks of non-C++ data into their code. Of course, `#include` expects the format of the data to be source code, and thusly the program fails with spectacular lexer errors. Thusly, many different tools and practices were adapted to handle this, as far back as 1995 with the `xxd` tool. Many industries need such functionality, including (but hardly limited to):

- Financial Development
 - representing coefficients and numeric constants for performance-critical algorithms;
- Game Development
 - assets that do not change at runtime, such as icons, fixed textures and other data;
 - Shader and scripting code;
- Embedded Development
 - storing large chunks of binary, such as firmware, in a well-compressed format;
 - placing data in memory on chips and systems that do not have an operating system or file system;
- Application Development
 - compressed binary blobs representing data
 - non-C++ script code that is not changed at runtime;
- Server Development

- configuration parameters which are known at build-time and are baked in to set limits and give compile-time information to tweak performance under certain loads;
- SSL/TLS Certificates hard-coded into your executable (requiring a rebuild and potential authorization before deploying new certificates), and;
- Static Analyzers
 - Static analyzers suffer -- much like their binary code generating friends -- from having to parse extremely large array literals;
 - Reduces memory pressure and enables better information tracking and potential sanitization (file source is not lost in build system).

In the pursuit of this goal, these tools have proven to have inadequacies and contribute poorly to the C++ development cycle as it continues to scale up for larger and better low-end devices and high-performance machines, bogging developers down with menial build tasks and trying to cover-up disappointing differences between platforms. It also absolutely destroys state-of-the-art compilers due to the extremely high memory overhead of producing an Abstract Syntax Tree for a braced initializer list of several tens of thousands of integral constants with numeric values at 255 or less.

The request for some form of `#include_string` or similar dates back quite a long time, with one of the oldest stack overflow questions asked-and-answered about it dating back nearly 10 years. Predating even that is a plethora of mailing list posts and forum posts asking how to get script code and other things that are not likely to change into the binary.

This paper proposes `<embed>` to make this process much more efficient, portable, and streamlined.

§ 4. Scope and Impact

`template <typename T = byte> constexpr span<const T> embed( string_view resource_identifier )` is an extension to the language proposed entirely as a library construct. The goal is to have it implemented with compiler intrinsics, builtins, or other suitable mechanisms. It does not affect the language. The proposed header to expose this functionality is `<embed>`, making the feature entirely-opt-in by checking if either the proposed feature test macro or header exists.

§ 5. Design Decisions

`<embed>` avoids using the preprocessor or defining new string literal syntax like its predecessors, preferring the use of a free function in the `std` namespace. This gives `std::embed` a greater degree of power and advantage over `<embed>`'s design is derived heavily from community feedback plus the rejection of the prior art up to this point, as well as the community needs demonstrated by existing practice and their pit falls.

§ 5.1. Implementation Experience & Current Practice

Here, we examine current practice, their benefits, and their pitfalls. There are a few cross-platform (and not-so-cross-platform) paths for getting data into an executable. We also scrutinize the performance, with numbers for both [memory overhead](#) and [speed overhead](#) available at the repository that houses the [current implementation](#). For ease of access, the numbers as of January 2020 with the latest versions of the indicated compilers and tools are replicated below.

All three major implementations were explored, plus an early implementation of this functionality in GCC. A competing implementation in a separate C++-like meta language called [Circle](#) was also looked at by the behest of Study Group 7.

§ 5.1.1. Speed Results

Below are timing results for a file of random bytes using a specific strategy. The file is of the size specified at the top of the column. Files are kept the same between strategies and tests.

- Intel Core i7-6700HQ @ 2.60 GHz
- 24.0 GB RAM 2952 MHz
- Debian Sid or Windows 10

— Method: Gather timings from `time` *nix program or `Measure-Command { ... }` PowerShell, compute mean

Strategy	4 bytes	40 bytes	400 bytes	4 kilobytes
#embed GCC	0.201 s	0.208 s	0.207 s	0.218 s
phd::embed GCC	0.709 s	0.724 s	0.711 s	0.715 s
xxd-generated GCC	0.225 s	0.215 s	0.237 s	0.247 s
xxd-generated Clang	0.272 s	0.275 s	0.272 s	0.272 s
xxd-generated MSVC	0.204 s	0.229 s	0.209 s	0.232 s
Circle @array	0.353 s	0.359 s	0.361 s	0.361 s
Circle @embed	0.199 s	0.208 s	0.204 s	0.368 s
objcopy (linker)	0.501 s	0.482 s	0.519 s	0.527 s

Strategy	40 kilobytes	400 kilobytes	4 megabytes	40 megabytes
#embed GCC	0.236 s	0.231 s	0.300 s	1.069 s
phd::embed GCC	0.705 s	0.713 s	0.772 s	1.135 s
xxd-generated GCC	0.406 s	2.135 s	23.567 s	225.290 s
xxd-generated Clang	0.366 s	1.063 s	8.309 s	83.250 s
xxd-generated MSVC	0.552 s	3.806 s	52.397 s	Out of Memory
Circle @array	0.353 s	0.363 s	0.421 s	0.585 s
Circle @embed	0.238 s	0.199 s	0.219 s	0.368 s
objcopy (linker)	0.500 s	0.497 s	0.555 s	2.183 s

Strategy	400 megabytes	1 gigabyte
#embed GCC	9.803 s	26.383 s
phd::embed GCC	4.170 s	11.887 s
xxd-generated GCC	Out of Memory	Out of Memory
xxd-generated Clang	Out of Memory	Out of Memory
xxd-generated MSVC	Out of Memory	Out of Memory
Circle @array	2.655 s	6.023 s
Circle @embed	1.886 s	4.762 s
objcopy (linker)	22.654 s	58.204 s

§ 5.1.2. Memory Size Results

Below is the peak memory usage (heap usage) for a file of random bytes using a specific strategy. The file is of the size specified at the top of the column. Files are kept the same between strategies and tests.

— Intel Core i7-6700HQ @ 2.60 GHz

— 24.0 GB RAM 2952 MHz

— Debian Sid or Windows 10

— Method: `/usr/bin/time -v` or Execute command hundreds of times, stare at Task Manager

Strategy	4 bytes	40 bytes	400 bytes	4 kilobytes
#embed GCC	17.26 MB	17.26 MB	17.26 MB	17.27 MB
phd::embed GCC	38.82 MB	38.77 MB	38.80 MB	38.80 MB
xxd-generated GCC	17.26 MB	17.26 MB	17.26 MB	17.27 MB
xxd-generated Clang	35.12 MB	35.22 MB	35.31 MB	35.88 MB
xxd-generated MSVC	< 30.00 MB	< 30.00 MB	< 33.00 MB	< 38.00 MB
Circle @array	53.56 MB	53.60 MB	53.53 MB	53.88 MB
Circle @embed	33.35 MB	33.34 MB	33.34 MB	33.35 MB
objcopy (linker)	17.32 MB	17.31 MB	17.31 MB	17.31 MB

Strategy	40 kilobytes	400 kilobytes	4 megabytes	40 megabytes
#embed GCC	17.26 MB	17.96 MB	53.42 MB	341.72 MB
phd::embed GCC	38.80 MB	40.10 MB	59.06 MB	208.52 MB
xxd-generated GCC	24.85 MB	134.34 MB	1,347.00 MB	12,622.00 MB
xxd-generated Clang	41.83 MB	103.76 MB	718.00 MB	7,116.00 MB

<code>xxd</code> -generated MSVC	~48.60 MB	~477.30 MB	~5,280.00 MB	Out of Memory
Circle <code>@array</code>	53.69 MB	54.73 MB	65.88 MB	176.44 MB
Circle <code>@embed</code>	33.34 MB	33.34 MB	39.41 MB	113.12 MB
<code>objcopy</code> (linker)	17.31 MB	17.31 MB	17.31 MB	57.13 MB

Strategy	400 megabytes	1 gigabyte
<code>#embed</code> GCC	3,995.34 MB	9,795.31 MB
<code>phd::embed</code> GCC	1,494.66 MB	5,279.37 MB
<code>xxd</code> -generated GCC	Out of Memory	Out of Memory
<code>xxd</code> -generated Clang	Out of Memory	Out of Memory
<code>xxd</code> -generated MSVC	Out of Memory	Out of Memory
Circle <code>@array</code>	1,282.34 MB	3,199.28 MB
Circle <code>@embed</code>	850.40 MB	2,128.36 MB
<code>objcopy</code> (linker)	425.77 MB	1,064.74 MB

§ 5.1.3. Results Analysis

The above clearly demonstrates the superiority of `std::embed` over latest optimized trunk builds of various compilers. It is also notable that originally the Circle language did not have an `@embed` keyword, but it was added in December 2019. When the compiler author was spoken to about Study Group 7's aspirations for a more generic way of representing data from a file, the ultimate response was this:

I'll add a new `@embed` keyword that takes a type and a file path and loads the file and embeds it into an array prvalue of that type. This will cut out the interpreter and it'll run at max speed. Feed back like this is good. This is super low-hanging fruit.

– Sean Baxter, [December 12th, 2019](#)

It was Circle's conclusion that a generic API was unsuitable and suffered from the same performance pitfalls that currently plagued current-generation compilers today. And it was SG7's insistence that a more generic API would be suitable, modeled on Circle's principles. Given that thorough exploration of the design space in Circle led to the same conclusion this proposal is making, and given the wide variety of languages providing a similar interface (D, Nim, Rust, etc.), it is clear that a more generic API is not desirable for functionality as fundamental and simple as this. This does not preclude a more generic solution being created, but it does prioritize the "Bird in the Hand" approach that the Direction Group and Bjarne Stroustrup have advocated for many times.

Furthermore, inspecting compiler bug reports around this subject area reveal that this is not the first time GCC has suffered monumental memory blowup over unoptimized representation of data. In fact, this is a [16+ year old problem that GCC has been struggling with for a long time now](#) (C++ version [here](#)). That the above numbers is nearing the best that can be afforded by some of the most passionate volunteers and experts curating an extremely large codebase should be testament to how hard the language is this area for compiler developers, and how painful it is for regular developers using their tools.

Clang, while having a better data representation and more optimized structures at its disposal, is similarly constrained. With significant implementation work, they are deeply constrained in what they can do:

It might be possible to introduce some sort of optimized representation specifically for initializer lists. But it would be a big departure from existing AST handling. And it wouldn't really open up new use cases, given that string literal handling is already reasonably efficient.

– [Eli Friedman, December 29th 2019](#)

Is this really the best use of compiler developer energy?

To provide a backdrop against which a big departure from current AST handling in can be compared, an implementation of the built-in necessary for this proposal is -- for an experienced developer -- at most a few day's work in either GCC or Clang. Other compiler engineers have reported similar ease of implementation and integration. Should this really be delegated to Quality of Implementation that will be need to be solved N times over by every implementation in their own particularly special way? Chipping away at what is essentially a fundamental inefficiency required by C++'s inescapable

tokenization model from the preprocessor plus the sheer cost of an ever-growing language that makes simple constructs like a brace initializer list of integer constants expensive is, in this paper's demonstrated opinion, incredibly unwise.

§ 5.1.4. Manual Work

Many developers also hand-wrap their files in (raw) string literals, or similar to massage their data -- binary or not -- into a conforming representation that can be parsed at source code:

0. Have a file `data.json` with some data, for example:

```
{ "Hello": "World!" }
```

1. Mangle that file with raw string literals, and save it as `raw_include_data.h`:

```
R"json({ "Hello": "World!" })json"
```

1. Include it into a variable, optionally made `constexpr`, and use it in the program:

```
#include <iostream>
#include <string_view>

int main() {
    constexpr std::string_view json_view =
#include "raw_include_data.h"
    ;

    // { "Hello": "World!" }
    std::cout << json_view << std::endl;
    return 0;
}
```

This happens often in the case of people who have not yet taken the "add a build step" mantra to heart. The biggest problem is that the above C++-ready source file is no longer valid in as its original representation, meaning the file as-is cannot be passed to any validation tools, schema checkers, or otherwise. This hurts the portability and interop story of C++ with other tools and languages.

Furthermore, if the string literal is too big vendors such as VC++ will hard error [the build \(example from Nonius, benchmarking framework\)](#).

§ 5.1.5. Processing Tools

Other developers use pre-processors for data that can't be easily hacked into a C++ source-code appropriate state (e.g., binary). The most popular one is `xxd -i my_data.bin`, which outputs an array in a file which developers then include. This is problematic because it turns binary data in C++ source. In many cases, this results in a larger file due to having to restructure the data to fit grammar requirements. It also results in needing an extra build step, which throws any programmer immediately at the mercy of build tools and project management. An example and further analysis can be found in the [§ 8.1.1 Pre-Processing Tools Alternative](#) and the [§ 8.1.2 python Alternative](#) section.

§ 5.1.6. `ld`, resource files, and other vendor-specific link-time tools

Resource files and other "link time" or post-processing measures have one benefit over the previous method: they are fast to perform in terms of compilation time. A example can be seen in the [§ 8.1.3 ld Alternative](#) section.

§ 5.1.7. The `incbin` tool

There is a tool called `[incbin]` which is a 3rd party attempt at pulling files in at "assembly time". Its approach is incredibly similar to `ld`, with the caveat that files must be shipped with their binary. It unfortunately falls prey to the same problems of cross-platform woes when dealing with VC++, requiring additional pre-processing to work out in full.

§ 5.2. Prior Art

There has been a lot of discussion over the years in many arenas, from Stack Overflow to mailing lists to meetings with the Committee itself. The latest advancements that had been brought to WG21's attention was [p0373r0 - File String Literals](#). It proposed the syntax `F"my_file.txt"` and `bF"my_file.txt"`, with a few other amenities, to load files at compilation time. The following is an analysis of the previous proposal.

§ 5.2.1. Literal-Based, `constexpr`

A user could reasonably assign (or want to assign) the resulting array to a `constexpr` variable as its expected to be handled like most other string literals. This allowed some degree of compile-time reflection. It is entirely helpful that such file contents be assigned to `constexpr`: e.g., string literals of JSON being loaded at compile time to be parsed by Ben Deane and Jason Turner in their CppCon 2017 talk, [constexpr All The Things](#).

§ 5.2.2. Literal-Based, Null Terminated (?)

It is unclear whether the resulting array of characters or bytes was to be null terminated. The usage and expression imply that it will be, due to its string-like appearance. However, is adding an additional null terminator fitting for desired usage? From the existing tools and practice (e.g., `xxd -i` or linking a data-dumped object file), the answer is no: but the syntax `bF"hello.txt"` makes the answer seem like a "yes". This is confusing: either the user should be given an explicit choice or the feature should be entirely opt-in.

§ 5.2.3. Encoding

Because the proposal used a string literal, several questions came up as to the actual encoding of the returned information. The author gave both `bF"my_file.txt"` and `F"my_file.txt"` to separate binary versus string-based arrays of returns. Not only did this conflate issues with expectations in the previous section, it also became a heavily contested discussion on both the mailing list group discussion of the original proposal and in the paper itself. This is likely one of the biggest pitfalls between separating "binary" data from "string" data: imbuing an object with string-like properties at translation time provide for all the same hairy questions around source/execution character set and the contents of a literal.

§ 5.3. Design Goals

Because of the aforementioned reasons, it seems more prudent to take a "compiler intrinsic"/"magic function" approach. The function overload takes the form:

```

template <typename T = byte>
constexpr span<const T> embed(
    u8string_view resource_identifier
);

template <size_t N, typename T = byte>
constexpr span<const T, N> embed(
    u8string_view resource_identifier
);

template <typename T = byte>
constexpr span<const T> embed(
    u8string_view resource_identifier, size_t limit
);

template <size_t N, typename T = byte>
constexpr span<const T, N> embed(
    u8string_view resource_identifier, size_t limit
);

```

`resource_identifier` is a `string_view` processed in an implementation-defined manner to find and pull resources into C++ at constexpr time. `limit` is the maximum number of elements the function call can produce (but it may produce less). The most obvious source will be the file system, with the intention of having this evaluated as a core constant expression. We do not attempt to restrict the `string_view` to a specific subset: whatever the implementation accepts (typically expected to be a relative or absolute file path, but can be other identification scheme), the implementation should use. The `N` template parameter is for specifying that the returned span has at least `N` elements (to fit in the statically-sized `span`).

§ 5.3.1. Implementation Defined

Calls such as `std::embed( "my_file.txt" );`, `std::embed( "data.dll" );`, and `std::embed<vertex>( "vertices.bin" );` are meant to be evaluated in a `constexpr` context (with "core constant expressions" only), where the behavior is implementation-defined. The function has unspecified behavior when evaluated in a non-constexpr context (with the expectation that the implementation will provide a failing diagnostic in these cases). This is similar to how include paths work, albeit `#include` interacts with the programmer through the preprocessor.

There is precedent for specifying library features that are implemented only through compile-time compiler intrinsics (`type_traits`, `source_location`, and similar utilities). Core -- for other proposals such as [p0466r1 - Layout-compatibility and Pointer-interconvertibility Traits](#) -- indicated their preference in using a `constexpr` magic function implemented by intrinsic in the standard library over some form of `template <auto X> thing { /* implementation specified */ value; };` construct. However, it is important to note that [\[p0466r1\]](#) proposes type traits, where as this has entirely different functionality, and so its reception and opinions may be different.

Finally, we use "implementation defined" so that compilers can produce implementation-defined search path for their translation units and modules during compilation with flags. The current implementation uses `-fembed-path=some/path/here/` to indicate this, but when it is standardized there will probably be an `-RI` or `-resource-include` flag instead. It is up to the implementation to pick what works for their platform.

§ 5.3.2. Binary Only

Creating two separate forms or options for loading data that is meant to be a "string" always fuels controversy and debate about what the resulting contents should be. The problem is sidestepped entirely by demanding that the resource loaded by `std::embed` represents the bytes exactly as they come from the resource. This prevents encoding confusion, conversion issues, and other pitfalls related to trying to match the user's idea of "string" data or non-binary formats. Data is received exactly as it is from the resource as defined by the implementation, whether it is a supposed text file or otherwise. `std::embed( "my_text_file.txt" )` and `std::embed( "my_binary_file.bin" )` behave exactly the same concerning their treatment of the resource.

§ 5.3.3. Constexpr Compatibility

The entire implementation must be usable in a `constexpr` context. It is not just for the purposes of processing the data at compile time, but because it matches existing implementations that store strings and huge array literals into a variable via `#include`. These variables can be `constexpr`: to not have a `constexpr` implementation is to leave many of the programmers who utilize this behavior without a proper standardized tool.

§ 5.3.4. Dependency-Scanning Friendly with `#depend`

One of the biggest hurdles to generating consensus was the deep-seated issues with dependency scanning. The model with only dealing with `#include` as a way of adding extra code or data-as-code means that all dependencies -- at the time of compiler invocation -- can be completely and statically known by the end of Phase 4 of compilation. This greatly aided in speedy dependency pulling. `std::embed` throws a wrench in that model as it can affect the generation of code at Phase 7 time, which is when `constexpr` is evaluated. By taking a `std::(u8)string_view`, it makes it impossible to know all files which may be used by a translation or module unit.

For this purpose, a new `#depend` preprocessor directive was introduced. It is intended to inform the compiler which translation units it depends on in order to make it simpler to retrieve all necessary dependencies. An advanced implementation can also replace all `#depend` directives with magical compiler builtins which instruct the compiler to cache the file's data as part of the translation unit.

This makes it possible to send minimal compiler reproductions and test cases for bug vetting, as well as allow distributed systems to continue to use the `-fdirectives-only` or `-frewrite-includes` flags (or similar preprocessor flags). For example, the GCC and Clang implementations linked above have `__builtin_init_array` and `__builtin_cache_array` methods that allow single translation units to be self-sufficient, while the compiler built-ins `__builtin_embed` will search the internally populated data cache from these entries in order to furnish data. This allows a complete system that keeps current tools working exactly as expected without any issues at all.

The `#depend` directive works by allowing a user to specify a *single-dependency* or a *family-dependency*. They are analogous to a single resource name or a "glob" resource name. These would, in file system terms, name a single file or all the files in a directory (subject to pattern matching). There used to also be a *"recursive-family-dependency"* for specifying searching for files in a directory, recursively. They looked like this:

```
// single-dependency directives
#depend <config/graph.bin>
#depend <foo.txt>
// family-dependencies
// do not "recurse" into directories
#depend "art/*"
#depend "art/mocks/*.json"
// recursive-family-dependency
// recurse through directories and similar
#depend "assets/**"
// mixed: all resources starting with
// "translation/", with all files that end in ".po",
// that have at least one "/" (one directory)
// after the "translation/", found recursively
#depend "translation/**/*.po"
```

Due to Windows being a pile of garbage for `FindFirstFile`-style directory iteration, it was voted to remove `**` as the recursive-family-dependency (see [§ 2 Relevant Polls](#)). Benchmarks for different styles of iteration may prove helpful here to convince others to maybe not do this.

`#depend` uses both the chevron-delimited `<foo.txt>` and the quote-delimited `"foo.txt"` formats. This is because one is for resources that are found only on the system / implementation paths similar to `-I` and `-isystem`, and then one that uses local look up plus the aforementioned resource paths. This can be useful for e.g. files that get embedded from system SDKs, like icons from Windows or Android build-time graphical resources or similar.

§ 5.3.5. Modules

Modules front-load and bring front-and-center one of the largest problems not with `std::embed`, but with `#depend`. `#depend` is a Compilation Phase 4 entity: that is, it does not exist beyond the preprocessor. In the world of `#include`, preprocessor commands were resolved that would not exist in the traditional `#include` world, or at least would not have to need immediate answers. In particular, the fact that modules can "save" Phase 4 information before proceeding with the rest of compilation opens interesting questions for Module Interface Units (AKA, the public-facing portion of modules). Module implementation units and other module-related bits are, thankfully, entirely unaffected.

The problem is illustrated most powerfully by the following snippet:

`m0.c++`:

```
export module m0;

#depend "header.bin"

import <embed>;
import combine;

export consteval auto f0(std::string_view f) {
    const auto h = std::embed("header.bin");
    const auto t = std::embed(f);
    return combine(h, t);
}
```

`m1.c++`:

```
export module m1;

#depend "default.stuff"

import m0;

export consteval auto f1(std::string_view f = "default.stuff") {
    return f0(f);
}

export consteval auto getpath() {
    return "default.stuff";
}
```

`translation_unit0.c++`:

```
import m1;
import print;
import <embed>

#depend "coolstuff.bin"

int main() {
    print(f1()); // [0] OK, fine
    print(f1("coolstuff.bin")); // [1] OK, good
    std::embed("header.bin"); // [2] Should this work?
    std::embed(getpath()); // [3] What about this?
}
```

In the line marked [0], everything is fine because the eventual call to `std::embed` originates from `m0`'s module interface unit, which can find `header.bin` and has an explicit dependency on `header.bin`. This is straight forward. Line [1] is fine as well, because there is an explicit `#depend` which makes the `coolstuff.bin` resource available from the current translation unit, which works with the `std::embed` call. The real problem arises, however, when we get to lines [2] and lines [3].

The way modules and macros work right now means macros do not go in (save from special implementation places, such as the command line/response file) and do not come out (unless explicitly exported). What, then, should happen to `#depend` and its preprocessor directives? Are the dependencies identified by `#depend` supposed to cascade through into translation units / modules that import them? This goes against the idea that modules can successfully isolate the preprocessor and suppress its stateful effects on downstream code.

Because the Committee voted to create a strong correlation between `std::embed` -- a Phase 7, regular C++ entity -- and `#depend` -- a Phase 4, preprocessor entity -- what used to be a compiler hint that the compiler can treat in any manner it so pleases and could instead gently warn on must now provide semantic meaning. Saying that lines [2] and [3] should work means acknowledging that `#depend` is a stateful preprocessor macro, which is something we have been trying to get rid of and reduce the usage of since the push for C++11. It also opens up the door to these questions in general regarding the usage, and requires compilers to have to answer these questions. If the answer becomes "yes", then programmers are back where we started several years ago with `#include` files and spending many people-months of time advocating for "`#depend` what you use". But even then, the answer to that question is even harder than before: because `std::embed` is a "real C++ entity" that exists outside of the preprocessor, things like default arguments and call location begin to start mattering a lot more than it should.

There are 4 ways to go about fixing this problem for `std::embed`:

0. Keep `#depend` as a requirement for finding files. Ban usage of the feature in module interface units. This makes it impossible to leak outside of implementation units or translation units because it is a `constexpr` entity.
1. Keep `#depend` as a requirement for finding files. Answer "no" to lines [2] and [3] above. This means that code that imports modules can no longer take an accidental dependency on things previous modules declared as their dependencies. It becomes less convenient if a path is provided by a library that needs to be used by `std::embed`, because then the user may have to duplicate `#depend` information.
2. Keep `#depend` as a requirement for finding files. Answer "yes" to lines [2] and [3] above. This means the `#depend` directive becomes entirely stateful and someone may accidentally rely on a dependency declaration in a module interface unit they use, and those module interface units can break downstream code by changing their `#depend` dependencies.
3. Remove the normative teeth of `#depend` for finding files. Do not let `#depend` leak out of modules. This follows the current module rules and does not create accidental library-user dependencies, while not impacting the usability of users who specify both `-resource-path=` flags and such appropriately.

#0 is way too big of a hammer to swing at this problem and drastically reduces the friendliness of the feature. It also taints the usability of both modules and `std::embed`, which is nowhere near a desirable outcome.

#1 seems like a decent solution, but still requires an excessive amount of client-side verbosity for individuals who write wrapper functions around `std::embed`. If a user relies on a function outside of their code to retrieve the path at `constexpr`-time, then they would need to duplicate whatever dependency-finding code the client has written as the user. Choosing this option makes the case for the string table / virtual file system approach in [§ 6.2 String Table / Virtual File System](#) more useful, since the fixed set of strings can be presented to the user in some fashion. (That is, if we conveniently ignore that recursive dependency specifications in any form still make this untenable.)

#2 introduces a heinous problem where users can be broken by library developers who rely on things local to their library but other developers "happen" to catch a hold of, making things miserable for both users and library developers. But, it does create a world in which simple things logically work even if a `std::embed` call is hidden inside of another `constexpr` function.

#3 makes it so a library and a user can have code that does not force a user to properly give all dependencies to outside build tools in this corner case in particular (at least, portably: warnings, warnings as errors, etc.). It does not require a strong coupling between the Phase 4 entity and the Phase 7 entity, eliminating the need for this question at all (and making it a matter of "can I, the compiler, find the file, or not").

This paper chooses #3. It takes the normative teeth out of `#depend` and leaves it to an implementation to warn when files are not explicitly depended upon. This means that a Phase 4 entity no longer has a tight coupling with a Phase 7 entity, keeping things separate. The modules space beyond Phase 4 is no longer complicated. Note that recompilation due to a change in any of the dependencies under this model still works without creating breaking problems in the program of a library user. If a used library's module interface unit has a `#depend` and a user changes the file that the module interface unit depends on, it triggers a recompile for that module interface unit. By triggering that recompilation, any down-stream

importer that depends on that module_ now must recompile. Now, how the compiler handles the `#depend` remains up to its implementation but it still has full reign to get that work done.

§ 5.3.6. Statically Polymorphic

While returning `std::byte` is valuable, it is impossible to `reinterpret_cast` or `bit_cast` certain things at compile time. This makes it impossible in a `constexpr` context to retrieve the actual data from a resource without tremendous boilerplate and work that every developer will have to do. While compile-time serialization is an important facet of this feature, there are many types where developers will end up just doing their own `bit_cast`-style `std::memcpy` of the bits to the target bits: to save on boilerplate, we instead offer the ability to do `std::embed<TYPE>`, where `TYPE` can be anything that satisfies trivial destructibility (e.g., `std::is_trivial_v<TYPE>` is `true`). This allows all C types and many types which are mirrored almost exactly in binary form to be pulled effortlessly into code.

§ 5.3.7. Optional Limit

Consider some file-based resources that are otherwise un-sizeable and un-seek/tellable in various implementations such as `/dev/urandom`. Telling the compiler to go fetch data from this resource infinitely can result in compiler lockups or worse: therefore, the user can specify an additional parameter to the function call such as `std::embed("/dev/infinity", 32);`. The 32 here is a `std::size_t` limit parameter in `std::embed`, and allows users to ask for up to but not more than limit elements in the returned `span`.

Note that as per § 5.3.6 Statically Polymorphic, the limit is specified in terms of `T`s, not bytes. This means `sizeof(T) * CHAR_BIT` bits are required, and the implementation is mandated to require up to but not more than that many.

Additionally, a user can provide a template argument `N` of type `std::size_t` to return a `span<T, N>`. This requires at least `N` elements, but maybe more can exist. One can use both of these to request "exactly this many" elements, e.g. a call `std::embed<32>("/dev/infinity", 32);` requires up to but not more than 32 elements (the limit parameter passed to the function) and that the returned span has at least 32 elements (the template parameter passed between the `<` and the `>`). Similarly, one could use `std::embed("/dev/infinity", 32).first<32>();`. This fully covers the design space for what making a subset of the data is currently used for (e.g., prefixed data in a common model format that then has "back references" to the data contained in the first `n` elements).

§ 5.3.8. UTF-8 Only

This is related to a serious problem for string literals, particularly those of Translation Phase 7. When a user types a string literal such as `"Fáuncy.softler"`, that string literal -- at Phase 5 of compilation -- gets translated from an idealized internal compiler character set to the "Execution Source Character Set". What this means is that the compiler is allowed to perform an implementation-defined translation from the string literal's ideal compiler representation to the execution character set (often, the presumed target execution character set). While this is not a problem for Clang -- which always uses UTF-8 -- and GCC -- which always uses UTF-8 unless someone specifically passes `-fexec-charset={something}` -- other compilers will take the string `"Fáuncy.softler"` and mangle it. This mangling does not have to come with warnings: in fact, MSVC will often times replace characters it cannot translate to the execution character set with either Mojibake or `"?"`.

The solution that Study Group 16 recommended was to allow `std::u8string_view` and ONLY `std::u8string_view` as the parameter type. Others in SG-16 voiced concern that this would hamper general usability, but `u8` string literals and `char8_t` were put in the standard for reasons exactly such as this. The execution character set is an unknown and often lossy encoding on legacy systems: requiring UTF-8 with `std::u8string_view` matches most internal compiler representations of idealized text storage and provides a lossless way to work with the resource system.

We note that it would be "maximally nice" to provide all of `std::[w|u8|u16|u32|string_view]` overloads to the table which would allow individuals working in the `constexpr` space their choice of presumed encoding to work in, but deem that the additional flexibility is likely not worth the additional overloads (even if those overloads are cheap to implement).

§ 6. Previous Implementations

This section is primarily to address feedback from polls wherein different forms and implementation strategies were asked for by the Evolution Working Group and other implementers. A tour of the design and implementation these cases helps show what has been considered.

§ 6.1. `#depend` Soft Warnings, Hard Errors?

The current specification makes it a hard error if a file has not been identified previously by a `#depend` directive. This makes the simple case of including a single file a bit more annoying than it needs to be, but also makes the case of general development with a non-distributed build system a pain. To be perfectly transparent, the author of this paper and almost 100% of the author's clients do not use distributed anything to build their code: errors at "file not found" are generally useful enough. Making the lack of matching `#depend` a hard error seems like pushing an important but nevertheless Committee-over-represented concern (i.e. distributed build tool users) onto all C++ users.

While the author feels a lot better about it being a soft warning that can be turned into a hard error by use of `-Werror-depend` for distributed build users, we leave the specification to strongly encourage implementations to hard error on an inability to not only find the resource but match it with a previous `#depend` declaration.

§ 6.2. String Table / Virtual File System

This implementation idea was floated twice, once during SG-7 discussion at the November 2019 Belfast meeting and again during the February 2020 Prague meeting. The crux of this version of `std::embed` is that it does not take resource identifiers related to e.g. the file system directly: a directive is used to build a "String Table" or "Translation Table" from system-specific paths to "friendly" names:

```
// map "foo.txt" to file name "foo"
#depend_name "foo.txt" "foo"

#ifdef _WIN32
    // map Windows-specific resource
    // "win/bazunga.bin" to file name "baz"
    #depend_name "win/bazunga.bin" "baz"
#else
    // map Unix-specific resource
    // "nix/bazunga.bin" to file name "baz"
    #depend_name "nix/bazooka.bin" "baz"
#endif

#include <embed>

int main () {
    // pulls foo.txt
    constexpr std::span<std::byte> foo = std::embed("foo");
    // pulls either bazunga or bazooka
    constexpr std::span<std::byte> baz = std::embed("baz");
    return foo[0] == 'f' && baz[2] == '\x3';
}
```

On the tin, this seems to bring nice properties to the table. We get to "erase" platform-specific paths and give them common names, we have a static list of names that we always pull from, and more. However, there are several approaches to this problem. Consider one of the primary use cases for `std::embed` as a `constexpr` function: reading a resource and, based on its contents, `std::embed`-ing other resources.

This becomes a problem: if `foo.txt` has a `get: bar.doc` line of text in it, we read that line from `foo.txt`, and then attempt to `std::embed("bar.doc")`, we get an error because we did not give `bar.doc` a string table name. This means we need to go back and not only mark it as a dependency with `#depend`, but also give it a static string-table based name.

Even conquering that problem, we face another: resource files -- JSON, SPIR-V, Wavefront OBJ, HLSL/GLSL, python, Lua, HSAIL, whatever -- do not speak "C++ String Table" to name their identifiers. Generally, these speak "File System":

we are adding a level of indirection that makes it `_impossible_` to keep working with the file system, especially when it comes to working with external resources. Most tools communicate and traffic interchange information VIA URIs or relative / local file paths. Forcing users to either adapt their code to play nice with the C++ file system or maintaining a translation table to and from "C++ String Table" to "C++ File System" is an order of magnitude additional complexity that is not both unnecessary and also painful.

There is absolutely room for a (potentially `constexpr`) Virtual File System in C++. This is not the feature that should bring us there. As the current implementation does, manipulating `-fembed-path=` similar to the way `#include` is handled alongside `-I` flags is a much better use of not only implementer time, but user time. It fits perfectly into the mental model ("include, but for resources") and lets users keep their file names as the unit of interchange as far as names go.

C++ does not need to make for itself a reputation of trying to be an extremely unique snowflake at the cost of usability and user friendliness.

§ 7. Changes to the Standard

Wording changes are relative to [\[n4842\]](#).

§ 7.1. Intent

The intent of the wording is to provide a function that:

- handles the provided resource identifying `string_view` in an implementation-defined manner;
- and, returns the specified `constexpr span` representing either the bytes of the resource or the bytes view as the type `T`.

The wording also explicitly disallows the usage of the function outside of a core constant expression by marking it `constexpr`, meaning it is ill-formed if it is attempted to be used at not-`constexpr` time (`std::embed` calls should not show up as a function in the final executable or in generated code). The program may pin the data returned by `std::embed` through the span into the executable if it is used outside a core constant expression.

§ 7.2. Proposed Feature Test Macro

The proposed feature test macros are `__cpp_lib_embed` for the library and `__cpp_pp_depend` for the preprocessor functionality.

§ 7.3. Proposed Wording

Append to §14.8.1 Predefined macro names `[cpp.predefined]`'s **Table 16** with one additional entry:

Macro name	Value
<code>__cpp_pp_depend</code>	<code>202006L</code>

Add a new section §15.4 Dependency `[cpp.depend]`:

15.4 Dependency [cpp.depend]

¹ A `#depend` directive establishes inputs or family of inputs upon which a translation unit depends.

² A preprocessing directive of the form

```
# depend < h-char-sequence > new-line
```

or

```
# depend " q-char-sequence " new-line
```

provides a dependency name. If an implementation does not find meaning in the quote-delimited *q-char-sequence*, it may reprocess this directive and treat it as a `#depend < h-char-sequence > new-line` directive using the same *q-char-sequence*, including any `<` or `>`.

³ The *q-char-sequence* or *h-char-sequence* may have one of 3 meanings, depending on the use of `*` or `**` within the sequence.

– If the sequence contains a `*` it denotes a *dependency-family*.

– Otherwise, it denotes a *single-dependency*.

⁴ [Example—

```
#depend "art.png" // this translation unit depends on 'art.png'
```

```
#depend <config/*.json> // this translation unit depends on all resources
                        // the implementation can find that
                        // end in ".json" and start with "config/".
```

```
#depend <data/*/*.bin> // this translation unit depends on all resources
                        // the implementation can find that
                        // end in ".bin", start with "data/"
                        // and contain a single "/" in-between.
```

— end Example].

⁵ Each of the *dependency-family* and *single-dependency* shall have an implementation-defined meaning which establishes search information for implementation-defined *resources* (e.g., for 19.20 [const.res]).

Append to §16.3.1 General [support.limits.general]'s Table 35 one additional entry:

Macro name	Value
<code>__cpp_lib_embed</code>	202006L

Append to §19.1 General [utilities.general]'s Table 38 one additional entry:

Subclause	Header(s)
19.20 Constant Resources	<code><embed></code>

Add a new section §19.20 Constant Resources [const.res]:

19.20 Constant Resources [const.res]

19.20.1 In general [const.res.general]

Constant resources allow the implementation to retrieve data from a variety of sources -- including implementation-defined places -- and allows their processing during constant evaluation.

19.20.2 Header <embed> synopsis [embed.syn]

```
namespace std {  
  
    template <typename T = byte>  
    constexpr span<const T> embed( string_view resource_identifier ) noexcept;  
  
    template <typename T = byte>  
    constexpr span<const T> embed( string_view resource_identifier, size_t limit ) no  
  
    template <::std::size_t N, typename T = byte>  
    constexpr span<const T, N> embed( string_view resource_identifier ) noexcept;  
  
    template <::std::size_t N, typename T = byte>  
    constexpr span<const T, N> embed( string_view resource_identifier, size_t limit )  
  
}
```

19.20.3 Function template embed [const.embed]

```
namespace std {  
  
    template <typename T = byte>  
    constexpr span<const T> embed( u8string_view resource_identifier ) noexcept;  
  
    template <typename T = byte>  
    constexpr span<const T> embed( u8string_view resource_identifier, size_t limit )  
  
    template <size_t N, typename T = byte>  
    constexpr span<const T, N> embed( u8string_view resource_identifier ) noexcept;  
  
    template <size_t N, typename T = byte>  
    constexpr span<const T, N> embed( u8string_view resource_identifier, size_t limit  
  
}
```

¹ Mandates: the implementation-defined bit size of the resource is a multiple of `sizeof(T) * CHAR_BIT` and `std::is_trivial_v<T>` is true. If the `limit` parameter is specified for the second or fourth overload, then then implementation-defined bit size of the resource must be less than or equal `limit * sizeof(T) * CHAR_BIT` and must be a multiple of `sizeof(T) * CHAR_BIT`. For the third and fourth overload, the implementation-defined bit size of the resource must be at least `sizeof(T) * CHAR_BIT * N`.

³ [*Note*— `std::is_trivial_v<T>` being true provides that types with non-trivial destructors do not need to be run for the implementation-provided static storage duration objects. — *end Note*].

⁴ If the implementation cannot find the resource specified after exhausting the sequence of implementation-defined search locations, or if the implementation finds the resource specified but that same resource was not named by a previous `#depend` directive in some manner, then the program is ill-formed.

⁵ *Returns*: A read-only view to a unique resource identified by the `resource_identifier` over a contiguous sequence of `T` objects with static storage duration. The mapping from the contents of the resource to the contiguous sequence of `T` objects is implementation-defined.

⁶ *Ensures*: For the second overload, let `r` denote the result of the function call. `r.size() <= limit` is true.

⁷ *Remarks*: The value of `resource_identifier` is used to search a sequence of implementation-defined places for a resource identified uniquely by `resource_identifier`. The mapping of the resource to the sequence of `T` is

implementation-defined. [*Note*— Implementations should provide a mechanism similar but not identical to `#include` (15.3 [cpp.include]) for finding the specified resource and in coordination with `#depend` (15.4 [cpp.depend]). — *end Note*]

§ 8. Appendix

§ 8.1. Alternative

Other techniques used include pre-processing data, link-time based tooling, and assembly-time runtime loading. They are detailed below, for a complete picture of today's sad landscape of options.

§ 8.1.1. Pre-Processing Tools Alternative

1. Run the tool over the data (`xxd -i xxd_data.bin > xxd_data.h`) to obtain the generated file (`xxd_data.h`):

```
unsigned char xxd_data_bin[] = {
    0x48, 0x65, 0x6c, 0x6c, 0x6f, 0x2c, 0x20, 0x57, 0x6f, 0x72, 0x6c, 0x64,
    0x0a
};
unsigned int xxd_data_bin_len = 13;
```

1. Compile `main.cpp`:

```
#include <iostream>
#include <string_view>

// prefix as constexpr,
// even if it generates some warnings in g++/clang++
constexpr
#include "xxd_data.h"
;

template <typename T, std::size_t N>
constexpr std::size_t array_size(const T (&)[N]) {
    return N;
}

int main() {
    static_assert(xxd_data_bin[0] == 'H');
    static_assert(array_size(xxd_data_bin) == 13);

    std::string_view data_view(
        reinterpret_cast<const char*>(xxd_data_bin),
        array_size(xxd_data_bin));
    std::cout << data_view << std::endl; // Hello, World!
    return 0;
}
```

Others still use python or other small scripting languages as part of their build process, outputting data in the exact C++ format that they require.

There are problems with the `xxd -i` or similar tool-based approach. Lexing and Parsing data-as-source-code adds an enormous overhead to actually reading and making that data available.

Binary data as C(++) arrays provide the overhead of having to comma-delimit every single byte present, it also requires that the compiler verify every entry in that array is a valid literal or entry according to the C++ language.

This scales poorly with larger files, and build times suffer for any non-trivial binary file, especially when it scales into Megabytes in size (e.g., firmware and similar).

§ 8.1.2. `python` Alternative

Other companies are forced to create their own ad-hoc tools to embed data and files into their C++ code. MongoDB uses a [custom python script](#), just to get their data into C++:

```
import os
import sys

def jsToHeader(target, source):
    outFile = target
    h = [
        '#include "mongo/base/string_data.h"',
        '#include "mongo/scripting/engine.h"',
        'namespace mongo {',
        'namespace JSFiles{',
    ]

    def lineToChars(s):
        return ','.join(str(ord(c)) for c in (s.rstrip() + '\n')) + ','

    for s in source:
        filename = str(s)
        objname = os.path.split(filename)[1].split('.')[0]
        stringname = '_jsgcode_raw_' + objname

        h.append('constexpr char ' + stringname + "[] = {")

        with open(filename, 'r') as f:
            for line in f:
                h.append(lineToChars(line))

        h.append("0};")
        # symbols aren't exported w/o this
        h.append('extern const JSFile %s;' % objname)
        h.append('const JSFile %s = { "%s", StringData(%s, sizeof(%s) - 1) };' %
            (objname, filename.replace('\\', '/'), stringname, stringname))

    h.append("} // namespace JSFiles")
    h.append("} // namespace mongo")
    h.append("")

    text = '\n'.join(h)

    with open(outFile, 'wb') as out:
        try:
            out.write(text)
        finally:
            out.close()

if __name__ == "__main__":
    if len(sys.argv) < 3:
        print "Must specify [target] [source] "
        sys.exit(1)
    jsToHeader(sys.argv[1], sys.argv[2:])
```

MongoDB were brave enough to share their code with me and make public the things they have to do: other companies have shared many similar concerns, but do not have the same bravery. We thank MongoDB for sharing.

§ 8.1.3. `ld` Alternative

A full, compilable example (except on Visual C++):

0. Have a file `ld_data.bin` with the contents `Hello, World!`.

1. Run `ld -r binary -o ld_data.o ld_data.bin`.
2. Compile the following `main.cpp` with `C++ -std=c++17 ld_data.o main.cpp`:

```
#include <iostream>
#include <string_view>

#ifdef __APPLE__
#include <mach-o/getsect.h>

#define DECLARE_LD(NAME) extern const unsigned char _section$__DATA_##NAME[];
#define LD_NAME(NAME) _section$__DATA_##NAME
#define LD_SIZE(NAME) (getsectbyname("__DATA", "__" #NAME)->size)

#elif (defined __MINGW32__) /* mingw */

#define DECLARE_LD(NAME) \
    extern const unsigned char binary_##NAME##_start[]; \
    extern const unsigned char binary_##NAME##_end[];
#define LD_NAME(NAME) binary_##NAME##_start
#define LD_SIZE(NAME) ((binary_##NAME##_end) - (binary_##NAME##_start))

#else /* gnu/linux ld */

#define DECLARE_LD(NAME) \
    extern const unsigned char _binary_##NAME##_start[]; \
    extern const unsigned char _binary_##NAME##_end[];
#define LD_NAME(NAME) _binary_##NAME##_start
#define LD_SIZE(NAME) ((_binary_##NAME##_end) - (_binary_##NAME##_start))
#endif

DECLARE_LD(ld_data_bin);

int main() {
    // impossible
    //static_assert(xxd_data_bin[0] == 'H');
    std::string_view data_view(
        reinterpret_cast<const char*>(LD_NAME(ld_data_bin)),
        LD_SIZE(ld_data_bin)
    );
    std::cout << data_view << std::endl; // Hello, World!
    return 0;
}
```

This scales a little bit better in terms of raw compilation time but is shockingly OS, vendor and platform specific in ways that novice developers would not be able to handle fully. The macros are required to erase differences, lest subtle differences in name will destroy one's ability to use these macros effectively. We omitted the code for handling VC++ resource files because it is excessively verbose than what is present here.

N.B.: Because these declarations are `extern`, the values in the array cannot be accessed at compilation/translation-time.

§ 9. Acknowledgements

A big thank you to Andrew Tomazos for replying to the author's e-mails about the prior art. Thank you to Arthur O'Dwyer for providing the author with incredible insight into the Committee's previous process for how they interpreted the Prior Art.

A special thank you to Agustín Bergé for encouraging the author to talk to the creator of the Prior Art and getting started on this. Thank you to Tom Honermann for direction and insight on how to write a paper and apply for a proposal.

Thank you to Arvid Gerstmann for helping the author understand and use the link-time tools.

Thank you to Tony Van Eerd for valuable advice in improving the main text of this paper.

Thank you to Lilly (Cpplang Slack, @lillypad) for the valuable bikeshed and hole-poking in original designs, alongside Ben Craig who very thoroughly explained his woes when trying to embed large firmware images into a C++ program for deployment into production. Thank you to Elias Kounen and Gabriel Ravier for wording review.

For all this hard work, it is the author's hope to carry this into C++. It would be the author's distinct honor to make development cycles easier and better with the programming language we work in and love. ♥

§ References

§ Informative References

[CIRCLE-EMBED-TWEET]

Sean Baxter. [@embed added to Circle](https://twitter.com/seanbax/status/1205195567003045888). URL: <https://twitter.com/seanbax/status/1205195567003045888>

[CLANG-LARGE-INIT-BUG]

LLVM Foundation. [Memory Consumption Reduction for Large Array Initialization?](https://bugs.llvm.org/show_bug.cgi?id=44399). URL: https://bugs.llvm.org/show_bug.cgi?id=44399

[CONSTEXPR-ALL-THE-THINGS]

Ben Deane; Jason Turner. [constexpr All The Things: CppCon 2017](https://www.youtube.com/watch?v=PJwd4JLYJJY). September 25th, 2017. URL: <https://www.youtube.com/watch?v=PJwd4JLYJJY>

[GCC-LARGE-INIT-BUG-C]

GCC. [\[8/9/10 regression\] Uses lots of memory when compiling large initialized arrays](https://gcc.gnu.org/bugzilla/show_bug.cgi?id=12245). URL: https://gcc.gnu.org/bugzilla/show_bug.cgi?id=12245

[GCC-LARGE-INIT-BUG-CPP]

GCC. [\[8/9/10 regression\] Uses lots of memory when compiling large initialized arrays](https://gcc.gnu.org/bugzilla/show_bug.cgi?id=14179). URL: https://gcc.gnu.org/bugzilla/show_bug.cgi?id=14179

[INCBIN]

Dale Weiler (graphitemaster). [incbin: load files at 'assembly' time](https://github.com/graphitemaster/incbin). URL: <https://github.com/graphitemaster/incbin>

[N4778]

Richard Smith. [Working Draft, Standard for Programming Language C++](https://wg21.link/n4778). 8 October 2018. URL: <https://wg21.link/n4778>

[N4842]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. [N4778 - Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4842.pdf). November 27th, 2019. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4842.pdf>

[NONIUS-VISUAL-C-ERROR]

R. Martinho Fernandes. [nonius generated HTML Reporter](https://github.com/libnonius/nonius/blob/devel/include/nonius/reporters/html_reporter.h%2B%2B#L42). September 1st, 2016. URL: https://github.com/libnonius/nonius/blob/devel/include/nonius/reporters/html_reporter.h%2B%2B#L42

[P0373R0]

Andrew Tomazos. [Proposal of File Literals](https://wg21.link/p0373r0). 21 May 2016. URL: <https://wg21.link/p0373r0>

[P0466R1]

Lisa Lippincott. [Layout-compatibility and Pointer-interconvertibility Traits](https://wg21.link/p0466r1). 12 February 2018. URL: <https://wg21.link/p0466r1>

[P1130]

JeanHeyd Meneide. [Module Resource Requirement Propagation](https://thephd.github.io/vendor/future_cxx/papers/d1130.html). November 26th, 2018. URL: https://thephd.github.io/vendor/future_cxx/papers/d1130.html